

Stacks & Queues in C++ From Two Rules to Placement-Ready

Everything for stacks, queues and deques: how to build them from scratch, the STL adapters, bracket and expression handling, the monotonic stack in full, the monotonic deque, the 12 problem patterns, and a curated problem list.

40

SECTIONS

12

PATTERNS

38

CODE BLOCKS

38

PROBLEMS

Prepared for **Hari** · on-campus placement preparation
Generated 08 September 2026

Contents

PART 0 — Stacks and queues, from absolute zero

- 0.1 The picture
- 0.2 The operations, and their exact names in C++
- 0.3 The vocabulary
- 0.4 Where you have already used a stack without noticing
- 0.5 How to recognise the topic in a problem statement

PART 1 — Building them yourself, and using the STL

- 1.1 A stack on top of an array
- 1.2 A stack on top of a linked list
- 1.3 A queue on a circular array — the one they ask about
- 1.4 A queue on a linked list
- 1.5 The STL — what to actually use in a contest or interview
- 1.6 Complexity, all in one place

PART 2 — The stack mental model

- 2.1 "The stack holds what is still unresolved"
- 2.2 Push indices, not values
- 2.3 The amortised argument you must be able to state
- 2.4 Recursion is a stack, and you can always convert

PART 3 — Brackets, and expressions

- 3.1 Balanced brackets — the canonical stack problem
- 3.2 The bracket family, and what changes each time
- 3.3 Infix, postfix, prefix — the three notations
- 3.4 The calculator family
- 3.5 Strings you process by collapsing

PART 4 — The monotonic stack

- 4.1 The idea
- 4.2 The four variants, as one table
- 4.3 Next Greater Element — write this from memory
- 4.4 Largest Rectangle in Histogram — the one to really understand

4.5 Trapping Rain Water — three ways to see it

4.6 The rest of the monotonic family

PART 5 — Queues, BFS, and the monotonic deque

5.1 The queue's real job is BFS

5.2 The monotonic deque — sliding window maximum

5.3 Queue-based streaming problems

5.4 Interconverting stacks and queues

PART 6 — Design problems

6.1 Min Stack — $O(1)$ minimum

6.2 The rest of the design family

PART 7 — The complete pattern taxonomy

How to tell patterns 5 and 6 apart

Choosing between a stack, a queue and a deque

PART 8 — How to approach ANY stack/queue problem

Step 1 — Say which of the two rules the problem wants

Step 2 — Answer the two design questions

Step 3 — Check whether it is secretly monotonic

Step 4 — Decide on the sentinel

Step 5 — Write the guards

Step 6 — Dry-run five inputs

PART 9 — The bug catalogue

PART 10 — Complexity cheat sheet

PART 11 — The problem list

PART 12 — A three-week plan

Quick reference card

PART 0 — Stacks and queues, from absolute zero

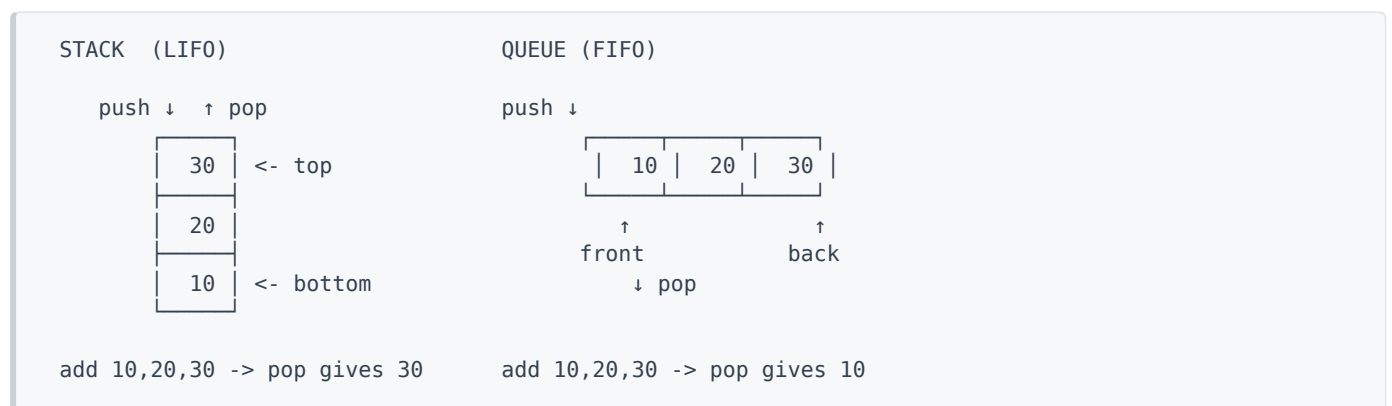
You already know a linked list and a tree. Both were about *shape* — how the nodes are wired together. Stacks and queues are different in kind: they are not new shapes, they are **rules about which element you are allowed to touch next**.

That is the whole topic. Two rules:

- A **stack** hands back the element you added **most recently**. Last In, First Out — **LIFO**.
- A **queue** hands back the element you added **first**. First In, First Out — **FIFO**.

Everything else in this guide — bracket matching, expression evaluation, the monotonic stack, sliding-window maximum, BFS — is a consequence of choosing one of those two rules.

0.1 The picture



A stack is a pile of plates: you can only take the top one. A queue is a line at a ticket counter: whoever arrived first is served first.

The single sentence to remember: a stack **reverses** the order of what goes through it; a queue **preserves** it. Every time you see a problem that says "reverse", "undo", "innermost first", "most recent", or "backtrack", the answer is a stack. Every time you see "in order", "level by level", "shortest number of steps", or "who came first", the answer is a queue.

0.2 The operations, and their exact names in C++

IDEA	STD::STACK	STD::QUEUE	COST
Add an element	<code>push(x)</code> / <code>emplace(x)</code>	<code>push(x)</code>	O(1)
Remove the next element	<code>pop()</code>	<code>pop()</code>	O(1)
Look at the next element	<code>top()</code>	<code>front()</code>	O(1)
Look at the other end	-	<code>back()</code>	O(1)
How many are in it	<code>size()</code>	<code>size()</code>	O(1)
Is it empty	<code>empty()</code>	<code>empty()</code>	O(1)

Two C++ facts that cause real bugs, and both of them will bite you in your first hour:

1. `pop()` returns nothing. In C++, `pop()` is `void`. You must *read* the value first and *then* remove it. This is unlike Python's `list.pop()` or Java's `Stack.pop()`, so if you learned either of those first, you will write this bug.

```
// WRONG — does not compile
int x = st.pop();

// RIGHT
int x = st.top();
st.pop();
```

The reason is exception safety: returning by value could throw while the container has already been modified, so the standard splits the two operations.

2. `top()` / `front()` / `pop()` on an empty container is undefined behaviour. Not an exception — undefined behaviour. It may print garbage, it may crash, it may appear to work on your machine and fail on the judge. **Guard every single access.**

```
if (!st.empty()) {           // this check is not optional
    int x = st.top();
    st.pop();
}
```

Nearly every "works locally, wrong answer on submission" story in this topic is one of those two.

0.3 The vocabulary

TERM	MEANING
Top	The stack end you push to and pop from
Bottom	The other end of a stack — unreachable without popping everything
Front	The queue end you remove from (the oldest element)
Rear / back	The queue end you add to (the newest element)
Overflow	Pushing onto a full fixed-size container
Underflow	Popping from an empty container — the undefined-behaviour case above
Deque	Double-ended queue: push and pop at both ends, all $O(1)$
Monotonic stack	A stack you deliberately keep sorted, by popping before you push (Part 4)
Amortised $O(1)$	Any single operation may be slow, but n operations together cost $O(n)$

0.4 Where you have already used a stack without noticing

The call stack. When you wrote `maxDepth(root)` for trees, the machine pushed a frame for every nested call and popped it on return. That is literally a stack, and it is why recursion depth costs memory. Part 2.4 shows how to convert any recursion into an explicit stack — which is exactly what iterative tree traversal was doing.

Undo in an editor. Ctrl+Z gives you back the *most recent* action. LIFO.

The browser back button. Pages you visited, most recent first. LIFO. (Forward is a second stack.)

Function-call tracebacks, bracket matching in your IDE, expression evaluation in the compiler — all stacks.

And queues: **print jobs, a keyboard buffer, task scheduling, message queues**, and, the one that matters for interviews, **breadth-first search**. The level-order traversal you wrote for trees was a queue; graph BFS in Topic 12 is the same queue.

0.5 How to recognise the topic in a problem statement

You will not be told "use a stack." You get these signals instead:

THE PROBLEM MENTIONS	REACH FOR
brackets, parentheses, valid/balanced, nesting	stack
"innermost", "matching pair", "undo the last"	stack
postfix, prefix, infix, calculator, evaluate an expression	stack
next greater / next smaller / previous greater / previous smaller	monotonic stack
histogram, rectangle, "span", "how many days until"	monotonic stack
trapping water, area under bars	monotonic stack or two pointers
sliding window maximum / minimum	monotonic deque
"level by level", "shortest number of steps", "minimum moves"	queue / BFS
"first non-repeating in a stream"	queue + counts
"design a structure with $O(1)$..."	stack/queue/deque + a second container
a string you must process and collapse ("remove adjacent duplicates")	stack

Part 7 turns this into the full 12-pattern taxonomy.

PART 1 — Building them yourself, and using the STL

Interviewers ask you to implement a stack or a queue from scratch surprisingly often — especially in Indian on-campus written rounds, where "implement a queue using a circular array" is a standing favourite. Write each of these once and you will never be caught out.

1.1 A stack on top of an array

The whole implementation is an array plus **one integer**: the index of the top.

```
class ArrayStack {
    static const int CAP = 1000;
    int data[CAP];
    int topIdx = -1;           // -1 means empty
public:
    bool empty() const { return topIdx == -1; }
    bool full() const { return topIdx == CAP - 1; }
    int size() const { return topIdx + 1; }

    void push(int x) {
        if (full()) throw overflow_error("stack overflow");
        data[++topIdx] = x;           // move up, then write
    }
    void pop() {
        if (empty()) throw underflow_error("stack underflow");
        --topIdx;                     // no need to erase, just forget it
    }
    int top() const {
        if (empty()) throw underflow_error("stack is empty");
        return data[topIdx];
    }
};
```

Everything is $O(1)$. Note `pop()` does not clear the slot — the element is simply out of reach, and the next `push` overwrites it. That is correct and it is what the real implementations do.

The `-1`-means-empty convention is worth adopting: it makes `push` a clean `data[++topIdx]` and `size()` a clean `topIdx + 1`. The alternative (`topIdx = 0` means empty, store at `data[topIdx++]`) also works; just do not mix the two halfway through, which is how off-by-one bugs are born.

1.2 A stack on top of a linked list

Push and pop at the **head** of a singly linked list — both $O(1)$, and there is no capacity limit.


```

class ListStack {
    struct Node { int val; Node* next; };
    Node* head = nullptr;
    int n = 0;
public:
    bool empty() const { return head == nullptr; }
    int size() const { return n; }

    void push(int x) { head = new Node{x, head}; n++; } // new node points at the old head

    void pop() {
        if (!head) throw underflow_error("stack underflow");
        Node* dead = head;
        head = head->next;
        delete dead;
        n--;
    }
    int top() const {
        if (!head) throw underflow_error("stack is empty");
        return head->val;
    }
    ~ListStack() { while (head) pop(); }
};

```

Why the head and not the tail? Insert-at-head and delete-at-head are $O(1)$ on a singly linked list; the tail costs $O(n)$ to delete because you cannot get to the previous node. This is the same reasoning you used in the linked-list topic — and it is the reason a stack is the *natural* structure for a singly linked list.

Array versus linked list, as an interview answer: the array version is faster (contiguous memory, cache-friendly, no allocation per element) but has a fixed capacity unless you grow it; the linked version grows freely but pays an allocation and a pointer per element and scatters through memory. `std::stack` uses neither — see 1.5.

1.3 A queue on a circular array — the one they ask about

A queue on a plain array is where beginners get stuck. Do the obvious thing — keep a `front` and a `rear` index and move both forward — and after a few pushes and pops the whole queue has drifted to the far end of the array, with unusable free space behind it:

after 3 pushes and 3 pops:

index:	0	1	2	3	4
	x	x	x		

↑
front = rear = 3

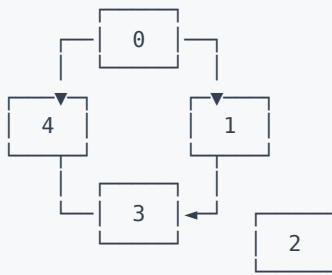
<- 3 slots wasted forever

The fix is to let the indices **wrap around** to 0 when they run off the end, turning the array into a ring. That is a circular queue, and modular arithmetic does the wrapping:

```

nextIndex = (index + 1) % capacity;

```



$(4 + 1) \% 5 == 0$ <- wraps

Now the awkward part, and the reason this question is asked: with `front` and `rear` alone, **a full queue and an empty queue look identical** — in both, `front == rear`. There are three standard fixes; keep a **count**, it is the one you will get right under pressure.

```

class CircularQueue {
    vector<int> data;
    int cap, front = 0, count = 0;           // count removes the full/empty ambiguity
public:
    explicit CircularQueue(int k) : data(k), cap(k) {}

    bool empty() const { return count == 0; }
    bool full() const { return count == cap; }
    int size() const { return count; }

    bool push(int x) {                       // enqueue at the rear
        if (full()) return false;
        int rear = (front + count) % cap;    // rear is derived, not stored
        data[rear] = x;
        count++;
        return true;
    }
    bool pop() {                             // dequeue from the front
        if (empty()) return false;
        front = (front + 1) % cap;           // just move the front forward
        count--;
        return true;
    }
    int getFront() const { return empty() ? -1 : data[front]; }
    int getRear() const { return empty() ? -1 : data[(front + count - 1) % cap]; }
};
  
```

Deriving `rear` from `front + count` instead of storing it separately means there is one less variable to keep in sync, and the full/empty question answers itself. This class is LeetCode 622, Design Circular Queue.

The other two fixes, in case an interviewer asks: **leave one slot always empty** (full becomes `(rear + 1) % cap == front`, at the cost of one wasted slot), or **store a boolean flag** for "the last operation was a push". Both work; both are fiddlier.

1.4 A queue on a linked list

Keep **both** a head and a tail pointer. Remove at the head, add at the tail — both $O(1)$.

```

class ListQueue {
    struct Node { int val; Node* next; };
    Node *head = nullptr, *tail = nullptr;
    int n = 0;
public:
    bool empty() const { return head == nullptr; }
    int size() const { return n; }

    void push(int x) { // enqueue at the tail
        Node* node = new Node{x, nullptr};
        if (tail) tail->next = node; // link the old tail forward
        else head = node; // first element: head too
        tail = node;
        n++;
    }
    void pop() { // dequeue at the head
        if (!head) throw underflow_error("queue underflow");
        Node* dead = head;
        head = head->next;
        if (!head) tail = nullptr; // <- the bug everyone forgets
        delete dead;
        n--;
    }
    int front() const {
        if (!head) throw underflow_error("queue is empty");
        return head->val;
    }
};

```

The classic bug, called out: when you pop the *last* element, `head` becomes `nullptr` but `tail` is left dangling at the freed node. The next `push` then writes through `tail->next` into freed memory. **Whenever `head` becomes null, null `tail` too.** This is bug #3 in Part 9.

1.5 The STL — what to actually use in a contest or interview

```

#include <stack>
#include <queue>
#include <deque>

stack<int> st;           st.push(1); st.top(); st.pop(); st.empty(); st.size();
queue<int> q;            q.push(1); q.front(); q.back(); q.pop();
deque<int> dq;           dq.push_front(1); dq.push_back(2);
                        dq.front(); dq.back(); dq.pop_front(); dq.pop_back();
                        dq[0]; // random access, unlike the adapters

```

`std::stack` and `std::queue` are **container adapters**, not containers: they wrap another container and hide everything except the legal operations. By default both wrap a `std::deque`.

	UNDERLYING CONTAINER	ITERABLE?	RANDOM ACCESS?
<code>stack<T></code>	<code>deque<T></code> by default	no	no
<code>queue<T></code>	<code>deque<T></code> by default	no	no
<code>deque<T></code>	itself	yes	yes, <code>dq[i]</code>
<code>vector<T></code>	itself	yes	yes

You can change the backing container, and occasionally you should — a `vector`-backed stack is measurably faster because it is contiguous:

```
stack<int, vector<int>> fast;    // legal and often quicker than the default
```

Practical advice for problems: use `std::stack` when you only ever need the top; use `std::deque` (or a plain `vector`) whenever you also need to **look inside**, iterate, or reach the bottom — which happens constantly in monotonic-stack problems, and `std::stack` will not let you. Many competitive programmers just use `vector` with `push_back` / `back` / `pop_back` for everything, and that is a perfectly good habit:

```
vector<int> st;
st.push_back(x);
int t = st.back();
st.pop_back();
if (!st.empty()) { /* ... */ }
for (int v : st) { /* you can iterate – a real stack cannot */ }
```

`std::queue` is fine as-is for BFS. Note also `std::priority_queue`, which looks like a queue but returns the **largest** element rather than the oldest — it is a heap, and it belongs to Topic 07, not here. Do not confuse the two: a `queue` is FIFO, a `priority_queue` is by-value order.

1.6 Complexity, all in one place

STRUCTURE	PUSH	POP	PEEK	RANDOM ACCESS	MEMORY
Array stack	O(1)	O(1)	O(1)	–	contiguous, fixed capacity
Linked stack	O(1)	O(1)	O(1)	–	one pointer per element
Circular-array queue	O(1)	O(1)	O(1)	–	contiguous, fixed capacity
Linked queue	O(1)	O(1)	O(1)	–	one pointer per element
<code>std::stack</code> / <code>std::queue</code>	O(1)	O(1)	O(1)	no	deque blocks
<code>std::deque</code>	O(1) both ends	O(1) both ends	O(1)	O(1)	blocks of elements
<code>std::vector</code> as a stack	amortised O(1)	O(1)	O(1)	O(1)	contiguous, doubles when it grows

Why `vector::push_back` is only *amortised* O(1): when the vector fills up it allocates a bigger block (usually double) and copies everything, which is O(n) for that one push. But doubling means that happens rarely enough that *n* pushes cost O(n) in total, so each one averages O(1). Being able to explain amortised analysis is worth real marks — you will need it again in 6.2 for the two-stack queue.

PART 2 — The stack mental model

Trees had one mental model — *handle this node, trust the children*. Stacks have one too, and it is just as load-bearing.

2.1 "The stack holds what is still unresolved"

Whenever you scan a sequence left to right and some items **cannot be finished yet**, park them on a stack. When the thing that resolves them arrives, pop.

That single sentence covers nearly the whole topic:

SCANNING...	WHAT IS UNRESOLVED	WHAT RESOLVES IT
a string of brackets	open brackets you have seen	the matching closing bracket
an expression	operators and operands waiting for precedence	a lower-precedence operator, or <code>)</code>
bar heights	bars that have not yet met a shorter bar	a bar shorter than them
temperatures	days still waiting for a warmer day	a warmer day
a directory path	folders you have descended into	<code>..</code>
an encoded string <code>3[ab]</code>	the repeat count and the text so far	the closing <code>]</code>

So the design question for almost every stack problem is exactly two questions:

1. **What do I push?** (usually an index, sometimes a value, sometimes a small pair or struct)
2. **When do I pop?** (the condition under which a parked item is now resolved)

Answer those two and you have written the solution. The loop around them is always the same shape:

```
for (int i = 0; i < n; i++) {
    while (!st.empty() && <the top is now resolved by element i>) {
        auto item = st.top(); st.pop();
        <use item and i together – this is where the answer is computed>
    }
    st.push(<something about i>);
}
```

Read that skeleton carefully. Almost every problem in Parts 3 and 4 is that exact loop with the two angle-bracketed lines filled in. Memorise the shape, not the problems.

2.2 Push indices, not values

A small habit that solves half the difficulty in this topic: **push the index `i`, not the value `a[i]`**.

The value is always recoverable as `a[st.top()]`, but the index is not recoverable from the value. And the moment a problem asks *how far away* the answer is — "how many days until a warmer

temperature", "how wide is this rectangle", "how much water sits between these bars" — you need the distance `i - st.top()`, which you can only compute if the stack holds indices.

```
st.push(i);           // do this
st.push(a[i]);        // not this, unless you are sure you never need distance
```

Daily Temperatures (LC 739) is the clean illustration: the answer is `i - poppedIndex`, and a value-based stack simply cannot produce it.

2.3 The amortised argument you must be able to state

Look at the skeleton in 2.1: there is a `while` loop inside a `for` loop. It looks like $O(n^2)$, and interviewers will ask.

It is $O(n)$. Here is the argument, and you should say it in this exact form:

Each element is pushed onto the stack exactly once and popped at most once. The inner `while` loop can therefore run at most n times in total across the entire outer loop, so the whole algorithm is $O(n)$.

That is called amortised analysis: you do not bound the work per iteration, you bound the total work over all iterations. Same reasoning as `vector::push_back` in 1.6. Being able to produce this argument on demand is a genuine differentiator in interviews — plenty of candidates write the right code and then cannot defend its complexity.

2.4 Recursion is a stack, and you can always convert

Every recursive function is using the machine's call stack. If you make that stack explicit, you get an iterative version — which never stack-overflows and which interviewers love to ask for.

```
// recursive
void preorder(TreeNode* n) {
    if (!n) return;
    visit(n);
    preorder(n->left);
    preorder(n->right);
}

// the same thing, with the stack written out by hand
void preorderIterative(TreeNode* root) {
    if (!root) return;
    stack<TreeNode*> st;
    st.push(root);
    while (!st.empty()) {
        TreeNode* n = st.top(); st.pop();
        visit(n);
        if (n->right) st.push(n->right);    // right first, so left pops first
        if (n->left) st.push(n->left);
    }
}
```

The push order is reversed because a stack reverses. That is the recurring correction whenever you hand-convert recursion.

Two classic exercises in the other direction — using recursion *on* a stack, with no extra container:

```

// Insert x at the correct sorted position in an already-sorted stack.
void insertSorted(stack<int>& st, int x) {
    if (st.empty() || st.top() <= x) { st.push(x); return; }
    int t = st.top(); st.pop();           // hold the top in the call frame
    insertSorted(st, x);
    st.push(t);                          // put it back on the way out
}

// Sort a stack using only recursion – a GfG / written-round favourite.
void sortStack(stack<int>& st) {
    if (st.empty()) return;
    int t = st.top(); st.pop();
    sortStack(st);
    insertSorted(st, t);
}

```

The trick in both is that the **call stack is the second container**. `int t` lives in a stack frame, so "no extra data structure" is technically honoured. Reversing a stack recursively is the same idea with `insertAtBottom` instead of `insertSorted`. Both are $O(n^2)$ time and $O(n)$ call-stack space — say so.

PART 3 — Brackets, and expressions

The first real family of stack problems. Master this and you have a third of the topic.

3.1 Balanced brackets — the canonical stack problem

Scan the string. Push every opening bracket. On a closing bracket, the top of the stack **must** be its partner.

```
bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else {
            if (st.empty()) return false;           // a closer with nothing open
            char open = st.top(); st.pop();
            if ((c == ')' && open != '(') ||
                (c == ']' && open != '[') ||
                (c == '}' && open != '{')) return false; // wrong partner
        }
    }
    return st.empty();                             // nothing left unclosed
}
```

Three checks, and dropping any one of them is a wrong answer:

CHECK	CATCHES	EXAMPLE THAT FAILS WITHOUT IT
<code>st.empty()</code> before popping	a closer with no opener	<code>)("</code>
the partner comparison	crossed brackets	<code>"([)]"</code>
<code>st.empty()</code> at the end	unclosed openers	<code>"(("</code>

A tidier version uses a map, which scales to more bracket types:

```
bool isValid(string s) {
    unordered_map<char, char> match{{'}, {'('}, {'}', {'['}, {'}', {'{'};
    stack<char> st;
    for (char c : s) {
        if (!match.count(c)) st.push(c);           // it is an opener
        else if (st.empty() || st.top() != match[c]) return false;
        else st.pop();
    }
    return st.empty();
}
```

The counter-only shortcut, and why it is a trap. With one bracket type you can drop the stack and just keep an integer, incrementing on `(` and decrementing on `)`, failing if it goes negative:

```
int bal = 0;
for (char c : s) { bal += (c == '(' ? 1 : -1); if (bal < 0) return false; }
return bal == 0;
```


That is correct for a *single* type and is the intended solution for LC 921 and LC 1541. It is **wrong the moment there are two or more bracket types**, because a counter cannot tell `"([)]"` from `"([)]"`. Know both and know which applies.

3.2 The bracket family, and what changes each time

Once the pattern clicks, an entire family opens up. In every one of them, **push the index rather than the character** so you can edit or measure the string afterwards.

PROBLEM	WHAT YOU PUSH	WHAT YOU DO AT THE END
20 Valid Parentheses	the opening char	require an empty stack
921 Minimum Add to Make Valid	- (counter suffices)	<code>unmatchedOpen + unmatchedClose</code>
1249 Minimum Remove to Make Valid	indices of unmatched <code>(</code>	mark every leftover index for deletion
32 Longest Valid Parentheses	indices, with a base of <code>-1</code>	<code>i - st.top()</code> gives the current valid length
856 Score of Parentheses	the running score of each nesting level	combine on <code>)</code>
1963 Balanced String Swaps	- (counter of unmatched <code>])</code>	<code>(unmatched + 1) / 2</code>

LC 32 Longest Valid Parentheses deserves a look, because the `-1` base is a genuinely clever trick worth stealing:

```
int longestValidParentheses(string s) {
    stack<int> st;
    st.push(-1); // sentinel: the index just before the current valid run
    int best = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        if (s[i] == '(') st.push(i);
        else {
            st.pop();
            if (st.empty()) st.push(i); // try to match
            else best = max(best, i - st.top()); // no match: i is the new base
        } // length from the base to here
    }
    return best;
}
```

The stack holds the index of the **last unmatched character**, so `i - st.top()` is the length of the valid run ending at `i`. Seeding with `-1` means a run starting at index 0 measures correctly without a special case. That "push a sentinel so the first case is not special" idea comes back in the histogram problem in Part 4.

3.3 Infix, postfix, prefix — the three notations

This is examined constantly in Indian written rounds, and almost never on LeetCode. Learn the definitions and the two conversions.

NOTATION	ALSO CALLED	A + B * C LOOKS LIKE
Infix	-	a + b * c
Postfix	Reverse Polish (RPN)	a b c * +
Prefix	Polish	+ a * b c

Infix is what humans write, and it needs precedence rules and parentheses to be unambiguous. Postfix and prefix need **neither** — the order of the symbols alone determines the order of evaluation, which is why compilers and calculators convert to postfix internally.

Evaluating postfix is the easiest algorithm in this entire guide: push operands, and on an operator pop two, apply, push the result.

```
int evalRPN(vector<string>& tokens) {
    stack<int> st;
    for (string& t : tokens) {
        if (t == "+" || t == "-" || t == "*" || t == "/") {
            int b = st.top(); st.pop();           // SECOND operand pops FIRST
            int a = st.top(); st.pop();
            if (t == "+") st.push(a + b);
            else if (t == "-") st.push(a - b);
            else if (t == "*") st.push(a * b);
            else st.push(a / b);
        } else st.push(stoi(t));
    }
    return st.top();
}
```

The order trap: the first value you pop is the *right* operand. Get this backwards and `+` and `*` still pass every test while `-` and `/` silently fail. Write `b` first, then `a`, then `a - b` — and check it on `["3", "4", "-"]`, which must give `-1`, not `1`.

Infix → postfix (the shunting-yard algorithm). Operands go straight to the output; operators wait on a stack until something of lower or equal precedence arrives.

```
int prec(char c) { return (c=='^') ? 3 : (c=='*' || c=='/') ? 2 : (c=='+' || c=='-') ? 1 : 0; }

string infixToPostfix(const string& s) {
    string out; stack<char> st;
    for (char c : s) {
        if (isdigit(c)) out += c;           // operand -> output
        else if (c == '(') st.push(c);
        else if (c == ')') {
            while (!st.empty() && st.top() != '(') { out += st.top(); st.pop(); }
            st.pop();                       // discard the '('
        } else {
            // an operator
            while (!st.empty() && prec(st.top()) >= prec(c) && c != '^') {
                out += st.top(); st.pop();
            }
            st.push(c);
        }
    }
    while (!st.empty()) { out += st.top(); st.pop(); }
    return out;
}
```

The `c != '^'` guard exists because `^` is **right-associative**: `2^3^2` means `2^(3^2)`, so an equal-precedence `^` must *not* be popped. Every other operator here is left-associative. That detail is exactly what written-round examiners check.

Infix → prefix, in one line of instruction: reverse the string, swap every `(` with `)`, run infix→postfix, then reverse the result.

Evaluating prefix: scan **right to left**, push operands, and on an operator pop two — but now the *first* pop is the **left** operand. It is the mirror of postfix in every respect.

3.4 The calculator family

LeetCode's calculator problems evaluate infix directly, and they are worth working through because they combine everything above.

LC 227 Basic Calculator II — `+ - * /` with no parentheses. The insight: additive operators can be deferred, but multiplicative ones must be applied immediately. So push signed numbers, and when the *pending* operator was `*` or `/`, fold it into the top of the stack right away. The answer is the sum of the stack.

```
int calculate(string s) {
    stack<int> st;
    long num = 0;
    char op = '+'; // the operator BEFORE the current number
    for (int i = 0; i < (int)s.size(); i++) {
        char c = s[i];
        if (isdigit(c)) num = num * 10 + (c - '0');
        if ((!isdigit(c) && c != ' ') || i + 1 == (int)s.size()) {
            if (op == '+') st.push(num);
            else if (op == '-') st.push(-num); // subtraction = pushing a negative
            else if (op == '*') { int t = st.top(); st.pop(); st.push(t * num); }
            else { int t = st.top(); st.pop(); st.push(t / num); }
            op = c; num = 0; // remember this operator for the next number
        }
    }
    int sum = 0;
    while (!st.empty()) { sum += st.top(); st.pop(); }
    return sum;
}
```

Two things make this work and both are worth naming. First, `op` holds the operator that came *before* the number you just finished reading, not the one after it — that one-step delay is the whole design. Second, the `|| i + 1 == s.size()` clause flushes the final number, which has no operator after it to trigger the push.

LC 224 Basic Calculator — `+ -` and **parentheses**, no `*` or `/`. Different technique: keep a running `result` and a running `sign`, and when you meet `(`, push the *context* (the result so far and the sign in front of the bracket) so you can restore it at `)`.

```

int calculate(string s) {
    stack<int> st; // alternating: result, sign, result, sign, ...
    int result = 0, sign = 1, num = 0;
    for (char c : s) {
        if (isdigit(c)) num = num * 10 + (c - '0');
        else if (c == '+') { result += sign * num; num = 0; sign = 1; }
        else if (c == '-') { result += sign * num; num = 0; sign = -1; }
        else if (c == '(') { st.push(result); st.push(sign); result = 0; sign = 1; }
        else if (c == ')') {
            result += sign * num; num = 0;
            result *= st.top(); st.pop(); // the sign in front of the '('
            result += st.top(); st.pop(); // the result before the '('
            sign = 1;
        }
    }
    return result + sign * num; // flush the last number
}

```

The order of the two pops matters and mirrors the order of the two pushes — sign was pushed last, so it pops first.

LC 394 Decode String — `3[a2[c]]` → `"accc"`. Same "push the context" idea, with two stacks (or one stack of pairs):

```

string decodeString(string s) {
    stack<int> counts;
    stack<string> parts;
    string cur; int k = 0;
    for (char c : s) {
        if (isdigit(c)) k = k * 10 + (c - '0'); // multi-digit counts exist: "100[a]"
        else if (c == '[') { counts.push(k); parts.push(cur); k = 0; cur.clear(); }
        else if (c == ']') {
            string inner = cur;
            cur = parts.top(); parts.pop(); // restore the outer text
            int rep = counts.top(); counts.pop();
            while (rep-- > 0) cur += inner; // append the repeated block
        } else cur += c;
    }
    return cur;
}

```

The generalisation worth carrying away: whenever a structure nests, push the *state you are about to overwrite* on the way in and restore it on the way out. Brackets, calculators, decoded strings, directory paths, DFS on a graph — all the same move. That is what "the stack holds what is still unresolved" means in practice.

3.5 Strings you process by collapsing

A whole set of easy-to-medium problems reduces to "build the answer on a stack, and pop when the new character cancels the top." The `string` itself is the stack — `push_back` / `back` / `pop_back`.

```
// LC 1047 – remove all adjacent duplicate characters
string removeDuplicates(string s) {
    string st;
    for (char c : s) {
        if (!st.empty() && st.back() == c) st.pop_back();    // cancel
        else st.push_back(c);
    }
    return st;
}
```

PROBLEM

THE POP CONDITION

1047 Remove All Adjacent Duplicates

top equals the new character

1209 Remove Adjacent Duplicates II

a run of `k` equal characters completes (push `{char, count}` pairs)

844 Backspace String Compare

the new character is `'#'`

71 Simplify Path

the component is `".."`

2390 Removing Stars From a String

the new character is `'*'`

735 Asteroid Collision

signs oppose and the incoming one is bigger

402 Remove K Digits

the top digit is **greater** than the new one and you still have removals left

316 Remove Duplicate Letters

the top is greater **and** appears again later **and** is not already used

The last two are already monotonic stacks — which is Part 4.

PART 4 — The monotonic stack

This is the highest-value part of the guide. A monotonic stack turns a whole class of problems that look $O(n^2)$ into $O(n)$, and roughly a third of all medium/hard stack questions in placement drives are one of its four variants.

4.1 The idea

A **monotonic stack** is an ordinary stack that you deliberately keep sorted — increasing or decreasing from bottom to top — by **popping anything that would break the order before you push**.

Why is that useful? Because of what the popping *means*. Suppose the stack is increasing and a new element `a[i]` arrives that is smaller than the top. You pop the top — and at that exact moment you know two things about the popped element that you could not know before:

- `a[i]` is its next smaller element to the right, because nothing between them was smaller (or it would have been popped earlier), and
- the element now below it on the stack is its previous smaller element to the left.

One pop hands you both boundaries. That is the whole trick. Everything below is bookkeeping.

```
a = [2, 1, 5, 6, 2, 3]      keeping an INCREASING stack of indices

i=0  push 0                st = [2]
i=1  a[1]=1 < 2  -> pop 2    st = [ ]      <- 2's next smaller is 1
      push 1                st = [1]
i=2  5 > 1, push            st = [1,5]
i=3  6 > 5, push            st = [1,5,6]
i=4  a[4]=2 < 6  -> pop 6    st = [1,5]      <- 6's next smaller is 2, prev smaller is 5
      a[4]=2 < 5  -> pop 5    st = [1]        <- 5's next smaller is 2, prev smaller is 1
      push 4                st = [1,2]
i=5  3 > 2, push            st = [1,2,3]
```

Notice how much information falls out of six steps of a single pass.

4.2 The four variants, as one table

Every monotonic-stack problem is one of these four. The **only** things that change are the comparison operator and whether you scan left-to-right or right-to-left.

YOU WANT, FOR EACH I	SCAN	STACK ORDER (BOTTOM→TOP)	POP WHILE
Next greater to the right	left → right	decreasing	<code>a[st.top()] < a[i]</code>
Next smaller to the right	left → right	increasing	<code>a[st.top()] > a[i]</code>
Previous greater to the left	left → right	decreasing	<code>a[st.top()] <= a[i]</code>
Previous smaller to the left	left → right	increasing	<code>a[st.top()] >= a[i]</code>

The two right-hand ("next") variants read the answer **at the moment of popping**. The two left-hand ("previous") variants read the answer **after popping, from whatever is left on top**. Both directions

come out of one loop:

```
// For every i: nextSmaller[i] and prevSmaller[i], in ONE left-to-right pass.
vector<int> nextSmaller(n, n), prevSmaller(n, -1);    // n and -1 mean "none"
stack<int> st;                                       // increasing by value
for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.top()] > a[i]) {
        nextSmaller[st.top()] = i;                  // read at pop time
        st.pop();
    }
    prevSmaller[i] = st.empty() ? -1 : st.top();     // read after popping
    st.push(i);
}
```

That eight-line block is the single most reusable piece of code in this guide. Swap `>` for `<` and it computes next/previous **greater** instead. Learn it as one unit.

How to pick the stack order: you want *next smaller*, so the stack must contain elements **still looking for** something smaller — which means everything on it is bigger than what is below it, i.e. increasing. Say it to yourself as "*the stack holds the elements still waiting for their answer*", which is exactly the Part 2.1 mental model.

Strict versus non-strict (`>` versus `>=`) matters when there are duplicate values. The safe default: use a strict comparison on one side and non-strict on the other, so that among equal values exactly one is counted as "the boundary". This is what makes LC 907 (Sum of Subarray Minimums) come out right, and getting it wrong is bug #9 in Part 9.

4.3 Next Greater Element — write this from memory

```
// LC 496 / the general form. res[i] = the first element to the right that is greater, else -1.
vector<int> nextGreater(vector<int>& a) {
    int n = a.size();
    vector<int> res(n, -1);
    stack<int> st;                                // indices, values decreasing bottom -> top
    for (int i = 0; i < n; i++) {
        while (!st.empty() && a[st.top()] < a[i]) {
            res[st.top()] = a[i];                  // a[i] is the answer for the popped index
            st.pop();
        }
        st.push(i);
    }
    return res;                                   // whatever is left never found one: -1
}
```

Everything left on the stack at the end had no next greater element — which is why `res` is pre-filled with `-1` rather than assigned in a second loop.

LC 739 Daily Temperatures is the same function with `res[st.top()] = i - st.top()` instead: the *distance*, not the value. This is why you push indices (Part 2.2).

LC 503 Next Greater Element II makes the array **circular**. The universal trick for circular arrays: run the loop **twice** and take indices modulo `n`.

```

for (int i = 0; i < 2 * n; i++) {
    int cur = a[i % n];
    while (!st.empty() && a[st.top()] < cur) { res[st.top()] = cur; st.pop(); }
    if (i < n) st.push(i);           // only push during the first pass
}

```

The `if (i < n)` guard is essential — without it the second pass pushes duplicate indices and the stack never drains.

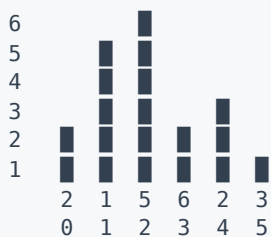
LC 901 Online Stock Span is *previous greater*, phrased as a streaming problem: the span is `i - prevGreaterIndex`. Because it is online, you keep the stack as a member variable between calls, and pushing `{price, span}` pairs lets you accumulate spans without storing indices at all.

4.4 Largest Rectangle in Histogram — the one to really understand

LC 84. Given bar heights, find the largest axis-aligned rectangle. It is asked at Amazon, Microsoft, Adobe and Goldman Sachs, and it is the parent problem of several others.

The key observation: every maximal rectangle is limited by some bar, and for *that* bar the rectangle extends left until a shorter bar and right until a shorter bar. So for each bar `i`, the widest rectangle of height `a[i]` has width `nextSmaller[i] - prevSmaller[i] - 1`. That is exactly the pair the block in 4.2 computes.

heights = [2, 1, 5, 6, 2, 3]



bar 2 (height 5): prevSmaller=1, nextSmaller=4
width = 4 - 1 - 1 = 2, area = 10
bar 3 (height 6): width = 1, area = 6
best = 10

The one-pass version computes both boundaries *at pop time*, which is tighter and is what you should write:

```

int largestRectangleArea(vector<int>& h) {
    h.push_back(0);           // sentinel: forces everything to drain
    stack<int> st;             // increasing heights
    int best = 0;
    for (int i = 0; i < (int)h.size(); i++) {
        while (!st.empty() && h[st.top()] > h[i]) {
            int height = h[st.top()]; st.pop();
            int left = st.empty() ? -1 : st.top(); // previous smaller, after popping
            best = max(best, height * (i - left - 1));
        }
        st.push(i);
    }
    h.pop_back();
    return best;
}

```

Two details that are the whole difficulty:

- **The 0 sentinel at the end.** Without it, bars still on the stack when the loop ends are never measured — an increasing input like `[1,2,3]` returns 0. Appending a zero-height bar guarantees

every bar gets popped. (Same idea as the `-1` sentinel in LC 32.)

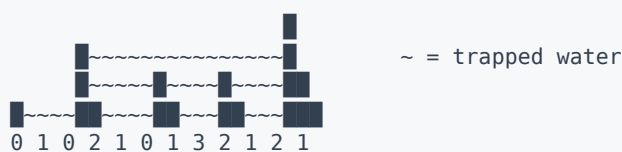
- `i - left - 1`, not `i - st.top()`. After the pop, `st.top()` is the previous *smaller* bar, which is **not** part of the rectangle — hence the `- 1`. And when the stack is empty the rectangle reaches all the way to index 0, which the `left = -1` convention handles without a special case.

LC 85 Maximal Rectangle — a binary matrix, find the largest all-ones rectangle — is this function called once per row. Build a running histogram where `heights[c]` is the number of consecutive 1s ending at the current row, then take the best over all rows. $O(\text{rows} \times \text{cols})$. Recognising that reduction is the entire problem.

4.5 Trapping Rain Water — three ways to see it

LC 42. This appears in more placement drives than any other array problem, and it has three legitimate solutions. Know at least two.

The observation behind all of them: the water above column `i` is `min(maxLeft[i], maxRight[i]) - a[i]`, floored at 0. Water is held by the *shorter* of the two walls.



Way 1 — prefix arrays. Precompute `maxLeft` and `maxRight` in two passes, then sum. $O(n)$ time, $O(n)$ space. This is the version to write first because it is impossible to get wrong.

Way 2 — two pointers. $O(1)$ space. Move whichever side has the smaller running maximum, because that side's answer is already determined by it.

```
int trap(vector<int>& a) {
    int l = 0, r = a.size() - 1, lMax = 0, rMax = 0, water = 0;
    while (l < r) {
        if (a[l] < a[r]) {
            lMax = max(lMax, a[l]);
            water += lMax - a[l];
            l++;
        } else {
            rMax = max(rMax, a[r]);
            water += rMax - a[r];
            r--;
        }
    }
    return water;
}
```

Way 3 — the monotonic stack. This is the one that belongs to this topic, and it fills water **layer by horizontal layer** rather than column by column. Keep a decreasing stack; when a taller bar arrives, the popped bar is the *floor* of a puddle bounded by the new bar on the right and by the new stack top on the left.

```

int trap(vector<int>& a) {
    stack<int> st;                                // decreasing heights
    int water = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!st.empty() && a[st.top()] < a[i]) {
            int floorIdx = st.top(); st.pop();
            if (st.empty()) break;                // no left wall -> no puddle
            int width = i - st.top() - 1;
            int height = min(a[st.top()], a[i]) - a[floorIdx];
            water += width * height;
        }
        st.push(i);
    }
    return water;
}

```

If asked which to use: say the two-pointer version, because it is $O(1)$ space, but explain the stack version to show you see the layered structure. Interviewers ask for "another way" constantly on this problem.

4.6 The rest of the monotonic family

PROBLEM	VARIANT	THE TWIST
496 Next Greater Element I	next greater	precompute for <code>nums2</code> , look up in a hash map
503 Next Greater Element II	next greater	circular — loop <code>2n</code> times, push only when <code>i < n</code>
739 Daily Temperatures	next greater	answer is the distance <code>i - st.top()</code>
901 Online Stock Span	previous greater	streaming — the stack persists between calls
84 Largest Rectangle	next + previous smaller	the <code>0</code> sentinel
85 Maximal Rectangle	84 per row	build a running histogram down the rows
42 Trapping Rain Water	next greater	fills water in horizontal layers
907 Sum of Subarray Minimums	next + previous smaller	count subarrays where <code>a[i]</code> is the minimum: <code>(i - prev) * (next - i)</code> ; use <code>>=</code> on one side only , or duplicates are double-counted
402 Remove K Digits	increasing stack	pop while the top is bigger and <code>k > 0</code> ; strip leading zeros; handle an empty result as <code>"0"</code>
316 Remove Duplicate Letters	increasing stack	pop only if the character appears again later and is not already in the result
456 132 Pattern	decreasing stack, scanned right to left	the popped values become a running "third" candidate
1019 Next Greater Node in Linked List	next greater	convert the list to a vector first — do not fight the pointers
2104 Sum of Subarray Ranges	max and min versions of 907	run the whole thing twice
735 Asteroid Collision	not strictly monotonic	the pop condition is a sign comparison, but the shape is identical

The counting idea in LC 907 is worth stating separately because it recurs: if you know, for each element, how far it extends left and right as the minimum, then it is the minimum of exactly `(i - prevSmaller[i]) * (nextSmaller[i] - i)` subarrays. Summing `a[i]` times that count over all `i` gives the answer in $O(n)$. Converting a "sum over all subarrays" question into a per-element contribution count is a standard interview move.

PART 5 — Queues, BFS, and the monotonic deque

5.1 The queue's real job is BFS

You have already used a queue for level-order traversal of a tree. That was breadth-first search, and it generalises directly to grids and graphs. The template never changes:

```
queue<Node> q;
q.push(start);
visited.insert(start);
int steps = 0;
while (!q.empty()) {
    int sz = q.size(); // freeze the level width - same trick as trees
    for (int i = 0; i < sz; i++) {
        Node cur = q.front(); q.pop();
        for (Node nxt : neighbours(cur)) {
            if (!visited.count(nxt)) { visited.insert(nxt); q.push(nxt); }
        }
    }
    steps++; // one full level = one step
}
```

Why BFS gives the shortest path (on an unweighted graph): nodes come off the queue in non-decreasing order of distance from the start, so the first time you reach a node is via a shortest path. That sentence is the interview answer.

`int sz = q.size()` captured **before** the inner loop is the same critical line as in tree level-order — read it inside and the levels smear together and your step count is wrong. It is bug #7 in Part 9.

Problems that are exactly this template: LC 994 Rotting Oranges (multi-source — push every rotten orange before starting), LC 542 01 Matrix, LC 1091 Shortest Path in Binary Matrix, LC 127 Word Ladder, LC 1926 Nearest Exit from a Maze. Graphs are Topic 12; what matters here is recognising that the queue is the engine.

5.2 The monotonic deque — sliding window maximum

LC 239, and the single most important queue problem for placements.

The problem: given an array and a window size `k`, report the maximum of every window as it slides. The naive answer is $O(n \cdot k)$. The required answer is $O(n)$.

The structure: a deque of **indices**, kept in decreasing order of value. It holds exactly the elements that could still be the maximum of some future window — the *candidates*.

Three rules, executed in this order for each new index `i`:

1. **Expire the front** if it has slid out of the window: `if (dq.front() <= i - k) dq.pop_front();`
2. **Dominate from the back:** while `a[dq.back()] <= a[i]`, pop the back. A smaller element that arrived *earlier* than `a[i]` can never be the max again — `a[i]` outlives it and beats it.
3. **Push `i`** at the back, then read the answer as `a[dq.front()]`.

```
vector<int> maxSlidingWindow(vector<int>& a, int k) {
    deque<int> dq; // indices, values decreasing
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        if (!dq.empty() && dq.front() <= i - k) dq.pop_front(); // 1. expire
        while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back(); // 2. dominate
        dq.push_back(i); // 3. push
        if (i >= k - 1) res.push_back(a[dq.front()]); // the front is the max
    }
    return res;
}
```

Why a **deque** and not a stack? Because you need to remove from *both* ends: the back for domination (the monotonic-stack move) and the front for expiry (the sliding-window move). That combination is the entire reason `std::deque` exists in your toolkit.

Complexity: $O(n)$ time by the same amortised argument as Part 2.3 — each index enters and leaves the deque once — and $O(k)$ space, since the deque never holds more than one window's worth.

The most common bug: using `<` instead of `<=` in the domination step, or storing values instead of indices. Store **indices** — you need them for the expiry check in step 1, and a value-based deque cannot do expiry at all.

Minimum instead of maximum: flip the comparison to `>=`. LC 1425 (Constrained Subsequence Sum) and LC 862 (Shortest Subarray with Sum at Least K) are the same deque applied to a prefix-sum array rather than to the values — that is the step up to hard.

5.3 Queue-based streaming problems

First non-repeating character in a stream (a GfG staple): keep a queue of characters in arrival order plus a frequency array. After each insertion, pop from the front while the front character's count exceeds 1; the new front is the answer, or `'#'` if the queue is empty.

```
queue<char> q;
int freq[26] = {0};
void insert(char c) {
    freq[c - 'a']++;
    q.push(c);
    while (!q.empty() && freq[q.front() - 'a'] > 1) q.pop();
}
char firstNonRepeating() { return q.empty() ? '#' : q.front(); }
```

LC 933 Number of Recent Calls — count pings in the last 3000 ms. Push the timestamp, pop from the front while `q.front() < t - 3000`, return `q.size()`. Three lines, and it is the purest illustration of a queue as a *sliding window over time*.

LC 346 Moving Average from Data Stream — the same shape with a running sum: push and add, and if the size exceeds the window, pop and subtract.

5.4 Interconverting stacks and queues

Both directions are standard interview questions, and the interesting part is the complexity analysis rather than the code.

Queue using two stacks (LC 232) — the good one. `in` receives pushes; `out` serves pops. Only refill `out` from `in` when `out` is empty, and refill it **completely**.

```
class MyQueue {
    stack<int> in, out;
    void shift() { while (!in.empty()) { out.push(in.top()); in.pop(); } }
public:
    void push(int x) { in.push(x); }
    int pop() { peek(); int v = out.top(); out.pop(); return v; }
    int peek() { if (out.empty()) shift(); return out.top(); }
    bool empty() { return in.empty() && out.empty(); }
};
```

Say this in the interview: "`push` is $O(1)$. `pop` is $O(n)$ in the worst case, but amortised $O(1)$ — every element is moved from `in` to `out` exactly once in its lifetime, so n operations cost $O(n)$ in total." The lazy refill (only when `out` is empty) is what makes the amortisation valid; refilling on every pop makes it genuinely $O(n)$ each time.

Stack using queues (LC 225) — you must make one operation $O(n)$; choose which. The tidiest is the single-queue "rotate on push": push the new element, then move every *previous* element from the front to the back, so the newest ends up at the front.

```
class MyStack {
    queue<int> q;
public:
    void push(int x) {
        q.push(x);
        for (int i = 1; i < (int)q.size(); i++) { q.push(q.front()); q.pop(); } // rotate
    }
    int pop() { int v = q.front(); q.pop(); return v; }
    int top() { return q.front(); }
    bool empty() { return q.empty(); }
};
```

$O(n)$ push, $O(1)$ pop, one container. Mention that the alternative — two queues with an $O(n)$ pop — is equally valid, and that the choice depends on which operation is more frequent.

PART 6 — Design problems

Design questions are stack/queue problems where the *interface* is the challenge: some operation must be $O(1)$ and the obvious implementation is not.

6.1 Min Stack — $O(1)$ minimum

LC 155. A stack that also answers `getMin()` in $O(1)$. The idea: a second stack that records, at every level, the minimum of everything at or below it.

```
class MinStack {
    stack<int> st, mins;
public:
    void push(int x) {
        st.push(x);
        mins.push(mins.empty() ? x : min(x, mins.top())); // push ALWAYS, even if not smaller
    }
    void pop() { st.pop(); mins.pop(); } // the two stay in lockstep
    int top() { return st.top(); }
    int getMin() { return mins.top(); }
};
```

Pushing onto `mins` unconditionally keeps the two stacks the same height, so `pop` is a single line and cannot desynchronise. The "optimisation" of pushing only when `x <= mins.top()` works too, but then `pop` must compare before popping `mins`, and using `<` instead of `<=` there breaks on duplicate minima — a classic planted bug.

The $O(1)$ -extra-space follow-up, if pressed: store the *encoded* value `2*x - min` when a new minimum arrives, and decode on pop. It needs `long long` to avoid overflow, and it is a party trick rather than good code. Say the two-stack version is what you would ship.

The same "shadow stack" idea gives a **max stack**, a **stack that reports its sum in $O(1)$** , and a **min queue** (which needs the monotonic deque from 5.2 instead).

6.2 The rest of the design family

PROBLEM	THE STRUCTURE	THE KEY IDEA
155 Min Stack	two stacks	shadow stack of running minima
232 Queue via stacks	two stacks	lazy refill → amortised $O(1)$
225 Stack via queues	one queue	rotate on push
622 Design Circular Queue	array + <code>front</code> + <code>count</code>	derive <code>rear</code> , count kills the full/empty ambiguity
641 Design Circular Deque	array + <code>front</code> + <code>count</code>	the same, with wrap-around at both ends
933 Number of Recent Calls	queue of timestamps	a sliding window over time
895 Maximum Frequency Stack	<code>freq</code> map + <code>map<int, stack<int>></code>	a stack per frequency level — pop from the highest
1381 Design a Stack With Increment	vector + a lazy <code>inc</code> array	defer the increment to pop time
146 LRU Cache	hash map + doubly linked list	not a queue, but the same "oldest out" instinct (Topic 15)

LC 895 Maximum Frequency Stack is the one worth internalising. `push(x)` increments `freq[x]` and pushes `x` onto `group[freq[x]]`. `pop()` takes from `group[maxFreq]`, and decrements `maxFreq` when that group empties. Keeping a separate stack per frequency level means ties are broken by recency automatically, with no comparator and no heap. That "index the containers by the quantity you are maximising" move is a genuinely reusable design idea.

PART 7 — The complete pattern taxonomy

Every problem in Part 11 is labelled with one of these twelve. When you can name the pattern, you already have most of the code.

#	PATTERN	RECOGNISE IT BY	CORE TOOL
1	Bracket matching	parentheses, valid, balanced, nesting	push openers, pop on the closer
2	Expression evaluation	infix / postfix / prefix, calculator	operand stack, operator stack
3	String collapse	"remove adjacent", backspace, stars, duplicates	the output string is the stack
4	Nested context save & restore	<code>k[...]</code> , parentheses in a calculator, paths	push the state you are about to overwrite
5	Monotonic stack — next/prev greater/smaller	"next greater", "how many days until", "span"	the 8-line block in 4.2
6	Monotonic stack — area & contribution	histogram, rectangle, trapping water, sum over all subarrays	<code>(i - prev) * (next - i)</code>
7	Monotonic deque	sliding window maximum / minimum	deque of indices, expire front, dominate back
8	Queue as a time window	"in the last X ms", moving average, first non-repeating	push, pop while the front is stale
9	BFS with a queue	shortest steps, level by level, spreading	freeze <code>q.size()</code> per level
10	Stack simulation	asteroid collision, car fleet, backspace-like rules	push, and pop while a collision rule holds
11	Design with an auxiliary structure	"all operations in O(1)"	a shadow stack, a second stack, or a map of stacks
12	Recursion ↔ explicit stack	"do it iteratively", "no recursion"	write the call stack out by hand

How to tell patterns 5 and 6 apart

They use the same stack; the difference is what you do **at the moment of the pop**.

- **Pattern 5** records a *single answer* for the popped element — its next greater value, or the distance to it. One number per element.
- **Pattern 6** computes a *region*: it combines the popped element with **both** the new index `i` and the element now exposed underneath, to get a width or a count. Two boundaries per element.

If the problem asks "what is the next X", you are in 5. If it asks about an area, a width, a span, or a sum over all subarrays, you are in 6 — and you will need `st.top()` **after** the pop as well as `i`.

Choosing between a stack, a queue and a deque

THE PROBLEM NEEDS	USE
only the most recent item	<code>stack</code> (or a <code>vector</code>)
only the oldest item	<code>queue</code>
both ends	<code>deque</code>
to look inside or iterate	<code>deque</code> or <code>vector</code> — not <code>std::stack</code>
the largest item regardless of arrival	<code>priority_queue</code> — that is a heap, Topic 07

PART 8 — How to approach ANY stack/queue problem

Six steps. They take about a minute and they will keep you out of the two traps that cost most marks — reaching for the wrong container, and writing an $O(n^2)$ simulation of something that has an $O(n)$ monotonic solution.

Step 1 — Say which of the two rules the problem wants

Read the statement and answer one question: **does the answer depend on the most recent thing, or on the oldest thing?** Most recent → stack. Oldest → queue. Both ends → deque. Do this before writing anything; picking the wrong container is a rewrite, not an edit.

Step 2 — Answer the two design questions

From Part 2.1:

1. **What do I push?** Default to the **index**. Push a value only when you are certain you will never need distance or position. Push a `pair` or a small struct when you need two things (value + count, price + span, character + run length).
2. **When do I pop?** Write the pop condition as an explicit boolean. This is the line that distinguishes the four monotonic variants, and it is where the strict-versus-non-strict decision lives.

Step 3 — Check whether it is secretly monotonic

Ask: *"is the brute force a nested loop that scans forward or backward looking for the first element bigger/smaller than this one?"* If yes, it is a monotonic stack and you can drop it to $O(n)$. The giveaway phrases are **next greater**, **next smaller**, **span**, **how many days until**, **the first taller/shorter**, **largest rectangle**, **how much water**.

For a sliding window with a fixed size asking for a max or min, it is a monotonic **deque**.

Step 4 — Decide on the sentinel

A large fraction of stack bugs are "the elements left on the stack when the loop ends were never processed." Two fixes:

- **Append a sentinel** to the input that forces a full drain — a `0` bar in the histogram, an infinite temperature, an empty token.
- **Or drain explicitly after the loop** with a second `while (!st.empty())`.

Also consider a sentinel at the *start*: pushing `-1` as a base index (LC 32) or treating an empty stack as index `-1` (LC 84) removes the "there is nothing to the left" special case entirely.

Step 5 — Write the guards

Before you write the body, write the guards, because they are what turns undefined behaviour into a correct answer:

```
while (!st.empty() && <condition>) { ... }      // empty check FIRST, always
if (st.empty()) { <the "no answer" case> }
```

C++ short-circuits `&&`, so `!st.empty()` on the left means `st.top()` is never evaluated on an empty stack. Writing the two operands in the other order is undefined behaviour and is bug #1.

Step 6 — Dry-run five inputs

1. **Empty input** — `""`, `[]`.
2. **One element.**
3. **Strictly increasing** input — `[1,2,3]`. This is the case where nothing ever pops during the loop, so it catches a missing final drain or sentinel.
4. **Strictly decreasing** input — `[3,2,1]`. Everything pops at once; catches wrong boundary arithmetic.
5. **All equal** — `[2,2,2]`. This is the one that catches a wrong strict/non-strict comparison, and it is the case people skip.

If those five pass on paper, submit.

PART 9 — The bug catalogue

Twelve bugs account for nearly every wrong answer in this topic. When a submission fails, read this list before re-reading your code.

- 1. Calling `top()` / `front()` / `pop()` without an empty check.** Undefined behaviour, not an exception. Symptom: passes locally, crashes or gives garbage on the judge. Always `!st.empty() && ...`, and always in that order.
 - 2. Expecting `pop()` to return the value.** It is `void` in C++. Read `top()` first, then `pop()`. Symptom: it does not compile — the friendliest bug on this list.
 - 3. Not nulling `tail` when a linked queue becomes empty.** The next `push` writes through a dangling pointer. Symptom: intermittent corruption, works on small tests.
 - 4. Full and empty look identical in a circular queue.** `front == rear` means both. Symptom: the queue silently overwrites elements or reports itself empty when it is full. Fix: keep a `count`.
 - 5. Popping operands in the wrong order for `-` and `/`.** The first pop is the **right** operand. Symptom: `+` and `*` work, `-` and `/` are wrong. Test on `["3", "4", "-"]` → `-1`.
 - 6. Using a counter instead of a stack with multiple bracket types.** Symptom: `"([)]"` is reported valid.
 - 7. Reading `q.size()` inside the level loop in BFS.** The children you just pushed join the current level. Symptom: step counts are wrong, everything collapses into one level. Freeze `int sz = q.size();` before the inner loop.
 - 8. Elements left on the stack when the loop ends are never processed.** Symptom: strictly increasing input returns 0 or misses answers. Fix: a sentinel, or an explicit drain loop.
 - 9. Wrong strict/non-strict comparison with duplicate values.** `>` versus `>=` in a monotonic stack. Symptom: correct on distinct values, wrong on `[2,2,2]` — double-counted or missed subarrays in LC 907, wrong widths in LC 84. Rule: use a strict comparison on one side and non-strict on the other so equal values are attributed to exactly one owner.
 - 10. Pushing values instead of indices.** You then cannot compute distance, width, or window expiry. Symptom: you get stuck halfway and cannot finish the formula. Fix: push `i` (Part 2.2).
 - 11. Off-by-one in the width formula.** It is `i - st.top() - 1` after the pop, not `i - st.top()`. The exposed stack top is the *boundary*, which is outside the region. Symptom: LC 84 and LC 42 are consistently one column too wide.
 - 12. Refilling the `out` stack on every pop in the two-stack queue.** Only refill when `out` is empty. Symptom: correct answers, O(n) per operation, TLE on the large tests — and the wrong answer to "what is the amortised complexity?"
- Two more worth naming:
- 13. Integer overflow in monotonic-stack sums.** LC 907's answer needs a modulus; LC 84's `height * width` can exceed `int` on adversarial input. Use `long long` when multiplying two input-sized quantities.

14. Forgetting to flush the last number in a calculator. The final digit run has no operator after it to trigger the push. Symptom: the last term of every expression is dropped. Fix: the `|| i + 1 == s.size()` clause, or a trailing `+` appended to the input.

PART 10 — Complexity cheat sheet

OPERATION / ALGORITHM	TIME	SPACE	NOTE
stack / queue / deque push, pop, peek	$O(1)$	-	<code>vector::push_back</code> is amortised $O(1)$
Valid parentheses	$O(n)$	$O(n)$	$O(1)$ with a counter, single bracket type only
Evaluate postfix	$O(n)$	$O(n)$	
Infix $\rightarrow$ postfix (shunting yard)	$O(n)$	$O(n)$	
Basic Calculator I / II	$O(n)$	$O(n)$	$O(1)$ space for II if you keep only the last term
Any monotonic stack pass	$O(n)$	$O(n)$	each element pushed once, popped once
Next greater / smaller, all i	$O(n)$	$O(n)$	the 8-line block in 4.2
Largest Rectangle in Histogram	$O(n)$	$O(n)$	
Maximal Rectangle ($m \times n$)	$O(m \cdot n)$	$O(n)$	histogram per row
Trapping Rain Water	$O(n)$	$O(n)$ stack / $O(1)$ two pointers	
Sliding Window Maximum	$O(n)$	$O(k)$	monotonic deque
BFS on a graph or grid	$O(V + E)$	$O(V)$	queue + visited set
Queue from two stacks	amortised $O(1)$	$O(n)$	worst case $O(n)$ for one pop
Stack from one queue	$O(n)$ push, $O(1)$ pop	$O(n)$	
Min Stack	$O(1)$ all operations	$O(n)$	shadow stack
Sort a stack recursively	$O(n^2)$	$O(n)$ call stack	

The sentences to say in interviews:

"Each element is pushed once and popped at most once, so even though there is a `while` inside the `for`, the total work is $O(n)$."

"`pop` is $O(n)$ in the worst case but amortised $O(1)$, because every element moves from the input stack to the output stack exactly once in its lifetime."

PART 11 — The problem list

Work these in order. Every one is solved in the companion **Stacks & Queues — Solved Questions** PDF, in exactly this order — but attempt each problem yourself first and only read the solution after you have genuinely struggled with it.

Tier 1 — Learn the mechanics (do all thirteen first)

#	PROBLEM	PATTERN
20	Valid Parentheses	1 — bracket matching
921	Minimum Add to Make Parentheses Valid	1 — the counter shortcut
155	Min Stack	11 — design, shadow stack
232	Implement Queue using Stacks	11 — design, amortised $O(1)$
225	Implement Stack using Queues	11 — design, rotate on push
622	Design Circular Queue	11 — design, the wrap-around
150	Evaluate Reverse Polish Notation	2 — expressions
682	Baseball Game	10 — simulation
1047	Remove All Adjacent Duplicates In String	3 — string collapse
844	Backspace String Compare	3 — string collapse
496	Next Greater Element I	5 — monotonic stack
739	Daily Temperatures	5 — monotonic stack, distances
933	Number of Recent Calls	8 — queue as a time window

Thirteen problems. When you can write all thirteen from a blank page, you have the mechanics. Do not move on before that is true.

Tier 2 — The placement core (these are what actually get asked)

#	PROBLEM	PATTERN
503	Next Greater Element II	5 — circular
901	Online Stock Span	5 — previous greater, streaming
84	Largest Rectangle in Histogram	6 — area ★
42	Trapping Rain Water	6 — area ★
239	Sliding Window Maximum	7 — monotonic deque ★
71	Simplify Path	3 — string collapse
394	Decode String	4 — nested context
227	Basic Calculator II	2 — expressions ★
946	Validate Stack Sequences	10 — simulation
402	Remove K Digits	5 — greedy monotonic
856	Score of Parentheses	4 — nested context
1249	Minimum Remove to Make Valid Parentheses	1 — indices of unmatched
32	Longest Valid Parentheses	1 — the <code>-1</code> sentinel
735	Asteroid Collision	10 — simulation
994	Rotting Oranges	9 — multi-source BFS

★ **84, 42, 239 and 227 are the four most-asked stack/queue questions in on-campus drives.** Amazon, Microsoft, Adobe, Goldman Sachs, TCS Digital, Infosys — all of them. Be able to write these four cold.

Tier 3 — Harder, but they do appear

#	PROBLEM	PATTERN
85	Maximal Rectangle	6 — histogram per row
224	Basic Calculator	4 — nested context
907	Sum of Subarray Minimums	6 — contribution counting
456	132 Pattern	5 — right-to-left monotonic
316	Remove Duplicate Letters	5 — greedy monotonic + lookahead
1019	Next Greater Node In Linked List	5 — monotonic on a list
636	Exclusive Time of Functions	4 — nested context
895	Maximum Frequency Stack	11 — a stack per frequency
862	Shortest Subarray with Sum at Least K	7 — deque on prefix sums
1425	Constrained Subsequence Sum	7 — deque + DP

The classic Indian-placement questions (GfG / written-round favourites)

These appear in written rounds and technical interviews, usually in GfG's wording rather than LeetCode's:

- Implement a stack using an array, and using a linked list
- Implement a queue using a circular array — **and explain the full/empty ambiguity**
- Implement a stack using two queues, and a queue using two stacks
- Implement **two stacks in one array**, and k stacks in one array
- Reverse a string / a stack / a queue using a stack
- **Sort a stack using recursion** (no other data structure)
- Insert an element at the bottom of a stack without another container
- Check for balanced parentheses; check for redundant brackets in an expression
- Infix $\rightarrow$ postfix, infix $\rightarrow$ prefix, postfix $\rightarrow$ infix, and evaluate all three
- Next greater element, next smaller element, previous greater, previous smaller
- **Stock span problem**
- Largest rectangle in a histogram; maximum area rectangle in a binary matrix
- Trapping rain water
- Sliding window maximum; minimum of all windows of every size
- Min stack / getMin in $O(1)$ time and $O(1)$ extra space
- Design a **LRU cache** (Topic 15, but it starts from this instinct)
- First non-repeating character in a stream
- Interleave the first half of a queue with the second half
- Reverse the first k elements of a queue

- Generate binary numbers from 1 to n using a queue
- Circular tour / petrol pump problem (a queue-shaped greedy)
- Celebrity problem (a stack-based elimination)
- The Josephus problem with a circular queue

"Two stacks in one array" is close to a guaranteed written-round question. The answer: grow the first stack from index 0 upward and the second from index `n-1` downward; they are full when the two tops meet, i.e. `top1 + 1 == top2`. Four lines, and it fails candidates constantly.

PART 12 — A three-week plan

Assumes about 1.5 hours a day. Stacks and queues are a smaller topic than trees — three weeks is enough if you are honest about Week 2.

Week 1 — The structures and the easy patterns. Implement all four containers from Part 1 by hand in one `.cpp` file: array stack, linked stack, circular queue, linked queue. Compile and test each. Then Tier 1's first eight: 20, 921, 155, 232, 225, 622, 150, 682. Finish the week with the string-collapse pair, 1047 and 844. Do not look at monotonic stacks yet.

Week 2 — The monotonic stack. This is the week that matters. Start with 496 and 739 and write the 8-line block from Part 4.2 from memory on three separate days until it is automatic. Then 503, 901, 402. Then spend two full days on **84 Largest Rectangle** and **42 Trapping Rain Water** — write 42 all three ways. Finish with 239 Sliding Window Maximum, which is the deque version of the same instinct.

Week 3 — Expressions, nesting and hardening. 227 and 224 back to back, then 394, 856, 71, 946, 735, 1249, 32. Add 994 for the BFS connection. Then Tier 3 as far as you get — 85 and 907 first, since they are direct extensions of what you already know. Finish by re-solving 84, 42, 239 and 227 on a timer, 20 minutes each, on paper, with no compiler.

Ongoing: every time a problem defeats you, log which of the 12 patterns it was and which of the 14 bugs you hit. After thirty problems that log tells you exactly what to drill.

Where this topic leads: the monotonic deque returns in sliding-window DP (Topic 11), the BFS queue is the engine of Topic 12 Graphs, and `priority_queue` in Topic 07 is the "queue" that orders by value instead of arrival. The recursion-to-stack conversion in Part 2.4 is what makes iterative DFS possible. Nothing here is wasted.

Quick reference card

```

// ===== the containers =====
stack<int> st;  st.push(x); st.top();  st.pop();  st.empty(); st.size();
queue<int> q;   q.push(x); q.front(); q.back(); q.pop();
deque<int> dq;  dq.push_front(x); dq.push_back(x); dq.front(); dq.back();
               dq.pop_front();  dq.pop_back();  dq[i];

// pop() returns void. top()/front() on an empty container is UNDEFINED BEHAVIOUR.

// ===== the universal stack skeleton =====
for (int i = 0; i < n; i++) {
    while (!st.empty() && <top is resolved by i>) {
        auto x = st.top(); st.pop();
        <compute the answer using x and i>
    }
    st.push(i);           // push the INDEX
}

// ===== next / previous smaller, in ONE pass =====
vector<int> nxt(n, n), prv(n, -1);
stack<int> st;           // increasing
for (int i = 0; i < n; i++) {
    while (!st.empty() && a[st.top()] > a[i]) { nxt[st.top()] = i; st.pop(); }
    prv[i] = st.empty() ? -1 : st.top();
    st.push(i);
}
// swap '>' for '<' to get next/previous GREATER

// ===== valid parentheses =====
for (char c : s) {
    if (isOpen(c)) st.push(c);
    else { if (st.empty() || st.top() != partner(c)) return false; st.pop(); }
}
return st.empty();

// ===== evaluate postfix =====
int b = st.top(); st.pop();           // RIGHT operand pops first
int a = st.top(); st.pop();
st.push(apply(op, a, b));

// ===== largest rectangle in a histogram =====
h.push_back(0);                      // sentinel forces the drain
for (int i = 0; i < h.size(); i++) {
    while (!st.empty() && h[st.top()] > h[i]) {
        int ht = h[st.top()]; st.pop();
        int left = st.empty() ? -1 : st.top();
        best = max(best, ht * (i - left - 1)); // note the -1
    }
    st.push(i);
}

// ===== sliding window maximum (monotonic deque) =====
for (int i = 0; i < n; i++) {
    if (!dq.empty() && dq.front() <= i - k) dq.pop_front(); // expire
    while (!dq.empty() && a[dq.back()] <= a[i]) dq.pop_back(); // dominate
    dq.push_back(i);
    if (i >= k - 1) res.push_back(a[dq.front()]);
}

// ===== BFS, level by level =====
q.push(start); seen.insert(start);
while (!q.empty()) {
    int sz = q.size();           // FREEZE the level width
    for (int i = 0; i < sz; i++) {
        auto cur = q.front(); q.pop();
        for (auto nxt : neighbours(cur))
            if (!seen.count(nxt)) { seen.insert(nxt); q.push(nxt); }
    }
    steps++;
}

```

```
// ===== circular queue: the two lines that matter =====
int rear = (front + count) % cap;           // derive rear, never store it
front = (front + 1) % cap;                 // dequeue

// ===== min stack =====
void push(int x) { st.push(x); mins.push(mins.empty() ? x : min(x, mins.top())); }
void pop()      { st.pop(); mins.pop(); }      // always in lockstep
```

The five rules, one last time:

1. **Most recent → stack. Oldest → queue. Both ends → deque.**
2. Answer the two questions before writing code: **what do I push, and when do I pop?**
3. **Push the index**, not the value.
4. Guard every access with `!st.empty() &&` — on the **left** of the `&&`.
5. A `while` inside a `for` is still **$O(n)$** : each element is pushed once and popped once. Say that sentence out loud in the interview.